

Reviewer Pack — Minimal Hodge Dimension vs. SAT Clause Set Entropy Correlati...

Entry f396eea2e602 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 05:28:30 UTC

Minimal Hodge Dimension vs. SAT Clause Set Entropy Correlation

Entry ID: f396eea2e602 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 05:28:30 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): SAT Clause Sets

Statement:

For every d -regular graph G , the minimal Hodge dimension ($\text{hd}(G)$) of its associated Tseitin formula φ_G is linearly correlated with its clause set entropy $H(\varphi_G)$, such that $\text{hd}(G) = \Theta(H(\varphi_G))$.

Rationale (proposer's reasoning):

Hodge theory provides a rich algebraic structure for studying geometric properties, which could potentially reveal nontrivial structural information about the complexity of SAT clause sets. A correlation between Hodge dimension and entropy might expose novel insights into the intractability of SAT.

Taxonomy category: Algebraic Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 80ef7fe485aba909

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given d -regular graph G , if the correlation coefficient between the minimal Hodge dimension ($\text{hd}(G)$) and clause set entropy ($H(\varphi_G)$) is within the range $[0.5, 1.0]$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebraic geometry AND hodge theory AND tseitin formula AND sat clause sets - minimal hodge dimension IN algebraic geometry AND related TO clause set entropy IN SAT clause sets - hodge theory AND Tseitin formulas AND correlation WITH sat clause set entropy

Top relevant hits considered: - [http://arxiv.org/abs/2102.03481v3] Noncommutative Hodge conjecture - [http://arxiv.org/abs/0707.3937v1] Cohomology theories for homotopy algebras and noncommutative geometry - [http://arxiv.org/abs/1602.06607v4] Why should one compute periods of algebraic cycles? - [http://arxiv.org/abs/2605.24303v1] Geometry of Neighborhoods of Minimal Rational Curves - [http://arxiv.org/abs/math/0106084v1] How to show a set is not algebraic - [http://arxiv.org/abs/1710.07476v6] Planar 3-SAT with a Clause/Variable Cycle - [http://arxiv.org/abs/hep-th/0203076v1] Extremal Black Holes in Supergravity and the Bekenstein-Hawking Entropy - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (d * n) % 2 != 0:
            return None
        graph = [[] for _ in range(n)]
        edges = set()
        for i in range(d * n // 2):
            u = random.randint(0, n - 1)
            v = random.randint(0, n - 1)
            while u == v or (u, v) in edges or (v, u) in edges:
                u = random.randint(0, n - 1)
                v = random.randint(0, n - 1)
            graph[u].append(v)
            graph[v].append(u)
            edges.add((u, v))
        return graph

    def tseitin_formula(graph):
```

```

clauses = []
literals = {}
literal_count = 0
for i in range(len(graph)):
    literals[i] = f'x{i}'
    literals[-i-1] = f'~x{i}'

for i, neighbors in enumerate(graph):
    clause = [literals[i+1]]
    for neighbor in neighbors:
        clause.append(literals[neighbor+1])
        clause.append(literals[-neighbor-1])
    clauses.append(clause)

for i in range(len(graph)):
    for j in range(i + 1, len(graph)):
        for k in range(j + 1, len(graph)):
            clause = [literals[i+1], literals[j+1], literals[k+1]]
            clause.append(literals[-i-1])
            clause.append(literals[-j-1])
            clause.append(literals[-k-1])
            clauses.append(clause)

return clauses

def hodge_dimension(graph):
    n = len(graph)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for i, neighbors in enumerate(graph):
        for neighbor in neighbors:
            adjacency_matrix[i][neighbor] = 1
            adjacency_matrix[neighbor][i] = 1

def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    pivot_col = 0
    for i in range(m):
        if pivot_col >= n:
            break
        max_row = i
        for j in range(i + 1, m):
            if abs(matrix[j][pivot_col]) > abs(matrix[max_row][pivot_col]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        if matrix[i][pivot_col] == 0:
            pivot_col += 1
            continue
        for j in range(i + 1, m):
            factor = matrix[j][pivot_col] / matrix[i][pivot_col]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
        rank += 1
        pivot_col += 1
    return rank

```

```

    return n - gaussian_elimination(adjacency_matrix)

def entropy(clauses):
    total_clauses = len(clauses)
    clause_counts = {}
    for clause in clauses:
        clause_str = tuple(sorted(clause))
        if clause_str in clause_counts:
            clause_counts[clause_str] += 1
        else:
            clause_counts[clause_str] = 1

    entropy_value = 0.0
    for count in clause_counts.values():
        probability = count / total_clauses
        entropy_value -= probability * math.log2(probability)

    return entropy_value

n_values = [10, 20, 30, 40]
results = []
for n in n_values:
    graph = generate_d_regular_graph(n, 3)
    if graph is None:
        continue
    clauses = tseitin_formula(graph)
    hd = hodge_dimension(graph)
    H = entropy(clauses)
    results.append({"n": n, "hd": hd, "H": H})

if not results:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n_max = max(result["n"] for result in results)
correlation_coefficient = sum((result["hd"] - mean_hd) * (result["H"] - mean_H) for result in results) / len(results)
mean_hd = sum(result["hd"] for result in results) / len(results)
mean_H = sum(result["H"] for result in results) / len(results)

return {
    "metric_name": "correlation",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": n_max,
    "conjecture_holds": 0.5 <= abs(correlation_coefficient) <= 1.0,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43]
results = []
for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")
    results.append(trial_result)

mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"])
std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_outside_range\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a412da73.py", line 153, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~^~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a412da73.py", line 119, in run_trial

```

    clauses = tseitin_formula(graph)
    ~~~~~^~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a412da73.py", line 48, in tseitin_formula

```

    clause.append(literals[neighbor+1])
    ~~~~~^~~~~~

```

KeyError: 10

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the correlation coefficient could not be calculated and thus the conjecture cannot be supported or falsified based on the provided information. | next: Re-run the test with a different seed to ensure it completes successfully and produces the necessary data for calculating the correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17543
2	propose	ollama_remote	glm4:latest	0	0	16940
3	propose	ollama_remote	glm4:latest	0	0	14484
4	preregistration	ollama_remote	glm4:latest	0	0	14118
5	novelty	ollama_remote	glm4:latest	0	0	14016
6	novelty	ollama_remote	glm4:latest	0	0	13545
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16032
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24114
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19649
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17759
11	judge	ollama_remote	glm4:latest	0	0	12166

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 180366 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/f396eea2e602.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f396eea2e602.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f396eea2e602.tar.gz` (if generated)